

TUTORIAL EXTENSO · NIVELES 1 A 6

ECOSISTEMAS DE OBJETOS

Herencia, Polimorfismo y Relaciones en Java

Este manual está diseñado bajo principios de **reducción de carga cognitiva** y pedagogía inmersiva. No aprenderás a hacer clases aisladas; aprenderás a orquestar sistemas profesionales donde cada pieza tiene un propósito, apoyado en ejemplos profundos, laboratorios y el arte de la asertividad escrita.

Herencia: Reutilización Inteligente

No repitas código. Si dos clases comparten el mismo ADN, extrae su lógica común a una clase padre y hereda el comportamiento.

LO QUE DESBLOQUEARÁS

1. Identificar la relación fundamental "Es un" (Is-A).
2. Utilizar la palabra reservada `extends` para heredar atributos y métodos.
3. Comprender el concepto de clase Base y clase Derivada.

PALABRAS CLAVE

`extends``protected``clase base``clase derivada``Is-A``reutilización`

1.1 | La Regla del "Es un"

JERARQUÍAS CLARAS

La herencia modela taxonomías reales. Un *Perro* es un *Animal*.
Un *Coche* es un *Vehículo*.

Cuando programamos, a menudo encontramos que múltiples entidades comparten características. En lugar de copiar y pegar los mismos atributos en diferentes clases (lo cual genera un mantenimiento de pesadilla), agrupamos lo común en una **Superclase** (o clase Padre) y hacemos que las **Subclases** (o clases Hijas) hereden de ella mediante la palabra clave `extends`.

Analicemos el caso de una biblioteca. Tienes Libros y Revistas. Ambos tienen un título y un código, pero solo el Libro tiene un número de páginas, y solo la Revista tiene un número de edición. El diseño correcto es crear una clase `Documento` de la cual hereden los demás.

HerenciaBasica.java

```
// 1. La Clase Padre (Superclase)
public class Documento {
    // 'protected' permite que las clases hijas accedan directamente a estos datos
    // Si fueran 'private', las hijas tendrían que usar getters/setters.
    protected String titulo;
    protected String codigo;

    public void mostrarInfoBasica() {
        System.out.println("Documento: " + titulo + " [" + codigo + "]");
    }
}

// 2. La Clase Hija (Subclase)
public class Libro extends Documento {
    private int numPaginas;
```

```
// ¡El libro acaba de heredar 'titulo', 'codigo' y 'mostrarInfoBasica'!
public void configurarLibro(String t, String c, int p) {
    this.titulo = t;          // Acceso permitido gracias a 'protected'
    this.codigo = c;
    this.numPaginas = p;
}

public void mostrarPaginas() {
    System.out.println("Páginas: " + numPaginas);
}
}
```

EL ERROR DE MALA HERENCIA

La regla de oro es **Is-A** (Es un). Un Guerrero **ES UN** Personaje (correcto). Un Motor **NO ES UN** Coche (incorrecto). Si usas `extends` solo para reusar código sin que exista una relación lógica "Es un", crearás un sistema frágil y muy difícil de mantener cuando el proyecto crezca.

LABORATORIO DE LA SESIÓN 1

Cimientos de E-commerce

TIEMPO ESTIMADO: 30 MINUTOS · PRÁCTICA INDIVIDUAL

Implementa una jerarquía básica para una tienda en línea:

1. Crea la clase `Producto` con los atributos `protected String nombre` y `protected double precio`. Añade un método `imprimirDetalle()`.
2. Crea la clase `Electrodomestico extends Producto` que añada `int mesesGarantia` y un método para mostrar la garantía.
3. En tu `Main`, instancia un `Electrodomestico`, asígnale valores a sus tres variables e invoca los métodos de impresión (el heredado y el propio).

Constructores y super()

Un objeto se construye de adentro hacia afuera. El padre siempre nace primero.

LO QUE DESBLOQUEARÁS

1. Entender el flujo de instanciación en Java.
2. Utilizar `super()` para invocar constructores padres.
3. Reutilizar métodos de la clase padre con `super.método()`.

PALABRAS CLAVE

`super()`

Constructor

`super.método`

Instanciación

2.1 | Construyendo como una Muñeca Rusa

En el Nivel 1 asignábamos valores a mano. En la vida real, usamos **Constructores**. Pero en la herencia hay una regla inquebrantable de Java: *No puedes construir un hijo sin antes haber construido a su padre*. Para lograr esto, Java nos provee la palabra clave `super`.

Empleados.java

```
public class Empleado {
    protected String nombre;
    protected double salarioBase;

    // Constructor del Padre
    public Empleado(String nombre, double salarioBase) {
        this.nombre = nombre;
        this.salarioBase = salarioBase;
        System.out.println("-> Nace el Empleado " + nombre);
    }

    public void trabajar() {
        System.out.println("Realizando labores generales.");
    }
}

public class Desarrollador extends Empleado {
    private String lenguajePpal;

    // Constructor del Hijo
    public Desarrollador(String nombre, double salario, String lenguaje) {
        // 1. Llamada OBLIGATORIA al constructor padre.
        // DEBE ser la primera línea del bloque.
        super(nombre, salario);
    }
}
```

```

// 2. Luego inicializamos lo que le pertenece solo al hijo
this.lenguajePpal = lenguaje;
System.out.println("-> El Empleado se especializa como Desarrollador");
}

// Sobrescritura de métodos (Override)
@Override
public void trabajar() {
    // Podemos reusar la lógica del padre llamando a super.metodo()
    super.trabajar();
    System.out.println("Escribiendo código en " + lenguajePpal);
}
}

```

DOS USOS DE SUPER

`super(...)`

Se usa **únicamente dentro de un constructor hijo** para llamar al constructor padre. Envía los datos que el padre necesita para inicializarse. Si no lo pones, Java intentará inyectar `super()` vacío (lo cual fallará si el padre no tiene un constructor por defecto).

`super.metodo()`

Se usa dentro de cualquier método para ejecutar la versión del padre de una función que hemos sobrescrito. Útil para **ampliar** comportamiento en lugar de reemplazarlo por completo.

LABORATORIO DE LA SESIÓN 2

Vehículos y Constructores

TIEMPO ESTIMADO: 40 MINUTOS · ENFOQUE: SUPER()

1. Crea `Vehiculo` con un constructor que reciba y asigne `marca` y `año`.
2. Crea `Automovil extends Vehiculo` con un constructor que reciba `marca, año, numPuertas`. Llama a `super(...)` correctamente.
3. Implementa `acelerar()` en el Vehículo y sobrescríbelo en el Automóvil usando `super.acelerar()` para complementar el mensaje.

Polimorfismo y Sobrescritura

Poly (muchas) + Morphos (formas). La capacidad de tratar a diferentes objetos hijos bajo el mismo lente de su padre, dejando que cada uno responda a su manera.

3.1 | Una Interfaz, Múltiples Comportamientos

EL VERDADERO PODER

El polimorfismo es lo que hace a la Programación Orientada a Objetos ser verdaderamente dinámica. El código se vuelve ciego al tipo exacto del hijo.

Supongamos que estamos haciendo un sistema para calcular pagos. Tienes `EmpleadoFijo` y `EmpleadoPorHoras`, ambos heredan de `Empleado`. Cada uno calcula su salario de forma distinta. Con el polimorfismo, podemos meterlos a todos en una sola lista de tipo `Empleado`, y al pedirles que calculen su salario, ¡cada uno sabrá exactamente cómo hacerlo sin que nosotros tengamos que preguntarles qué tipo de empleado son!

A este principio se le conoce como **Dynamic Binding** o enlace dinámico. Java decide en tiempo de ejecución a qué método llamar según la naturaleza real del objeto que creamos con `new`.

PolimorfismoEjemplo.java

```
// Clase Padre
public class Animal {
    public void hacerSonido() {
        System.out.println("...");
    }
}

// Clases Hijas
public class Perro extends Animal {
    @Override
    public void hacerSonido() {
        System.out.println("Guau! Guau!");
    }
}

public class Gato extends Animal {
    @Override
    public void hacerSonido() {
        System.out.println("Miau~");
    }
}

// La Magia del Polimorfismo en acción:
public class Main {
    public static void main(String[] args) {
        // UPCASTING: Usamos una variable del tipo Padre para guardar al Hijo
        Animal miMascota1 = new Perro();
        Animal miMascota2 = new Gato();

        // Arreglo polimórfico: Acepta cualquier cosa que sea un Animal
```

```
Animal[] zoologico = { miMascota1, miMascota2, new Animal() };

// Aquí ocurre la magia. Un solo ciclo, comportamientos distintos.
for (Animal a : zoologico) {
    a.hacerSonido(); // Java sabe cuál versión ejecutar
}
}
```

UPCASTING E INSTANCEOF

Al hacer `Animal a = new Perro();` hacemos un **Upcasting**. Es seguro porque todo `Perro` es *un* `Animal`. Sin embargo, al hacerlo, la variable `a` solo podrá acceder a los métodos definidos en `Animal`. Si `Perro` tuviera un método `perseguirCola()`, `a` no podría llamarlo directamente.

Para recuperar la forma original y llamar a métodos propios del hijo, debemos asegurarnos de quién es usando `instanceof` y luego hacer un **Downcasting**:

```
if (a instanceof Perro) { Perro p = (Perro) a;
p.perseguirCola(); }
```

LABORATORIO DE LA SESIÓN 3

Orquesta de Figuras

TIEMPO ESTIMADO: 45 MINUTOS · ENFOQUE: ARREGLOS POLIMÓRFICOS

1. Crea la clase `Figura` con un método `calcularArea()` que retorne `0.0`.
2. Crea `Circulo extends Figura` (añade atributo `radio`) y `Rectangulo extends Figura` (añade `base` y `altura`). Usa constructores con `super()`.
3. Sobrescribe `calcularArea()` en ambas con las fórmulas matemáticas correctas (Para círculo usa `Math.PI * radio * radio`).
4. En Main, crea un Array de `Figura` y mete círculos y rectángulos mezclados. Pásales un ciclo `for` que imprima el área de cada uno invocando el mismo método polimórfico.

Agregación: Relación "Tiene-un" Débil

Los objetos colaboran en equipo, pero cada uno mantiene su propia independencia y ciclo de vida.

4.1 | La Vida Independiente

EL TEST DEFINITIVO

Pregúntate: "Si el objeto contenedor se destruye, ¿el contenido sigue existiendo?" Si la respuesta es SÍ, es Agregación.

Dejamos atrás "Es un" (Herencia). Pasamos a las relaciones estructurales: **"Tiene un" (Has-A)**. La primera de ellas es la *Agregación*. Un `Departamento` universitario *tiene* un `Profesor`. Pero, si el Departamento de Física cierra por falta de presupuesto, el Profesor Albert Einstein no deja de existir; puede ir a investigar a otro lugar. Tienen ciclos de vida separados.

AgregacionEjemplo.java

```
class Profesor {
    private String nombre;
    private String especialidad;

    public Profesor(String nombre, String especialidad) {
        this.nombre = nombre;
        this.especialidad = especialidad;
    }

    public String getNombre() { return nombre; }
}

class Departamento {
    private String nombreDepto;
    // Agregación: El departamento TIENE un profesor como director
    private Profesor director;

    public Departamento(String nombreDepto) {
        this.nombreDepto = nombreDepto;
        // Nota fundamental: NO hacemos "new Profesor()" aquí adentro.
    }

    // Inyección de Dependencias: Recibimos al profesor desde afuera
    public void asignarDirector(Profesor p) {
        this.director = p;
        System.out.println(p.getNombre() + " dirige " + nombreDepto);
    }
}

public class Main {
    public static void main(String[] args) {
        // 1. Creamos al profesor (Existe por sí solo en memoria)
        Profesor prof = new Profesor("Alan Turing", "Computación");
    }
}
```

```
// 2. Creamos el departamento
Departamento depto = new Departamento("Ciencias Exactas");

// 3. Asociamos (Agregamos)
depto.asignarDirector(prof);
}
}
```

ANATOMÍA DE LA AGREGACIÓN

Declaración	Se define como un atributo del tipo de la clase colaboradora: <code>private Profesor director;</code>
Inyección	El objeto NO se instancia dentro de la clase que lo usa. Se crea externamente y se recibe a través de un setter (<code>asignarDirector</code>) o en el propio constructor por parámetro.
Acoplamiento	Bajo. Si mañana eliminas el objeto <code>depto</code> (el recolector de basura lo borra), la variable <code>prof</code> en el <code>main</code> sigue viva y contiene toda su información intacta.

LABORATORIO DE LA SESIÓN 4

Gestión de Flota

TIEMPO ESTIMADO: 40 MINUTOS · ENFOQUE: INDEPENDENCIA DE DATOS

Modelar un sistema donde los conductores sean asignados a autobuses, manteniendo su entidad separada.

1. Crea `Conductor` con atributos `nombre` y `licencia`.
2. Crea `Autobus` con `placa` y `ruta`.
3. Implementa la **Agregación**: `Autobus` debe tener un atributo `Conductor conductorAsignado`.
4. En `Autobus` añade el método `asignarConductor(Conductor c)`.
5. En el Main, crea dos conductores y un autobús. Asigna el primero, imprime el estado, y luego re-asigna el autobús al segundo conductor (simulando un cambio de turno).

Composición: Relación de Vida o Muerte

Cuando el "Todo" es el dueño absoluto de sus "Partes". Si la entidad principal desaparece, sus componentes lógicos dejan de existir de inmediato.

5.1 | El Acoplamiento Fuerte

DISEÑO ORGÁNICO

Una habitación no existe sin la casa que la contiene. Un corazón no late sin su cuerpo. Eso es la composición.

La **Composición** es la versión estricta de la Agregación. Aquí la relación "Tiene-un" es tan fuerte que las partes no pueden existir lógicamente sin el contenedor. La creación y destrucción del objeto interno es gestionada enteramente por el objeto principal.

Pensemos en un sistema para un e-commerce. Una `OrdenDeCompra` está compuesta por `ItemOrden`. Si un usuario cancela y elimina la Orden de Compra, los ítems individuales de esa orden (por ejemplo, la entrada "3 x Zapatos" *dentro de ese ticket específico*) no tienen razón de existir en la base de datos de forma aislada.

ComposicionEjemplo.java

```
import java.util.ArrayList;
import java.util.List;

// La "Parte"
class ItemOrden {
    private String producto;
    private int cantidad;

    public ItemOrden(String producto, int cantidad) {
        this.producto = producto;
        this.cantidad = cantidad;
    }

    public void mostrarItem() {
        System.out.println("-> " + cantidad + "x " + producto);
    }
}

// El "Todo" (Contenedor)
class OrdenDeCompra {
    private String idOrden;
    private List<ItemOrden> items; // Composición fuerte

    public OrdenDeCompra(String idOrden) {
        this.idOrden = idOrden;
        // 1. El contenedor crea y administra su colección de partes.
        this.items = new ArrayList<>();
    }

    // 2. Ocultamos la creación del Item al exterior
```

```

public void agregarProducto(String producto, int cantidad) {
    // ¡CLAVE! La clase principal hace el "new" de la parte
    ItemOrden nuevoItem = new ItemOrden(producto, cantidad);
    items.add(nuevoItem);
}

public void imprimirRecibo() {
    System.out.println("Orden: " + idOrden);
    for (ItemOrden item : items) {
        item.mostrarItem();
    }
}
}

```

EL SECRETO DE LA COMPOSICIÓN

Observa con atención el método `agregarProducto`. El método principal (el exterior) **NO** hace un `new ItemOrden()` y se lo pasa. El exterior solo pasa datos en bruto (`String producto, int cantidad`). **La propia clase contenedora (OrdenDeCompra) es la que usa `new` para crear la pieza interna.** De esta forma, se asegura que el objeto interno nace y muere bajo el control absoluto de la Orden.

LABORATORIO DE LA SESIÓN 5

Estructuras Dependientes (Casa y Habitaciones)

TIEMPO ESTIMADO: 50 MINUTOS · PRÁCTICA DE ENCAPSULAMIENTO

1. Crea `Habitacion` con atributos `nombre` (ej. "Cocina") y `metrosCuadrados`.
2. Crea `Casa` con un atributo `direccion` y una colección de Habitaciones (`ArrayList<Habitacion>`).
3. La clase `Casa` debe **componer** a las habitaciones. En tu clase `Main` está estrictamente prohibido hacer `new Habitacion()`.
4. Implementa el método `construirCuarto(String nombre, double mts)` dentro de `Casa` que instancie y guarde la habitación.
5. Crea un método `calcularMetrosTotales()` en `Casa` que use un ciclo for para sumar y retornar el área de todos sus cuartos.

Taller Final: "Cruce Código"

Integra Herencia, Polimorfismo, Agregación y Composición en un solo ecosistema de software realista, diseñado para gestionar movilidad y recursos en la ciudad.

APLICACIÓN REAL

El software real nunca usa un solo concepto. Los ingenieros mezclan patrones arquitectónicos para responder a la naturaleza exacta del problema de negocio.

Llegó el momento de probar que dominas el arte de conectar clases. Has explorado la teoría, has trazado el código mentalmente y has superado los laboratorios aislados. En este Taller Final, diseñarás el backend del proyecto de movilidad **Cruce Código para Armenia, Quindío**.

Este sistema permitirá gestionar diversos perfiles de usuarios que asocian sus vehículos personales y reservan cupos (espacios) en una plataforma de parqueo céntrica, integrando todos los pilares estructurales aprendidos.

Ecosistema de Movilidad: Cruce Código

MODALIDAD: PROYECTO · SUBIDA: GITHUB / MOODLE DE "PROGRAMACIÓN 1"

1. Diseña la Jerarquía (Herencia y Polimorfismo):

1. Crea una clase base `Usuario` con atributos `id` y `nombre`, y su constructor apropiado.
2. Crea las clases hijas: `Administrador` (añade `codigoAcceso`) y `Cliente` (añade `saldoReserva`). Implementa correctamente `super()` en ambos.
3. Crea un método `mostrarPerfil()` en `Usuario` y sobrescríbelo (`@Override`) en las hijas para imprimir detalles según el rol.

2. Asigna Independencia (Agregación):

1. Crea la clase `Vehiculo` con `placa` y `tipo` (moto/carro).
2. La clase `Cliente` debe poseer una relación de **Agregación** con `Vehiculo`. (La moto no muere si el usuario borra la app). Inyecta el vehículo mediante `public void registrarVehiculo(Vehiculo v)`.

3. Construye el Núcleo (Composición):

1. Crea la clase `EspacioParqueo` que guarde su identificador (ej: "A1") y su estado (libre/ocupado).
2. Crea la clase `PlataformaCentro`. En su constructor, debe hacer uso de **Composición** instanciando internamente una colección de 20 espacios de parking (usa un bucle `for` que pase "Cupo-1", "Cupo-2", etc.).

CÓMO PROBAR TU CÓDIGO (MAIN):

Crea un arreglo polimórfico de `Usuario[]` metiendo a un `Administrador` y a un `Cliente`. Recórrelo mostrando el perfil. Luego, crea un `Vehículo` suelto, inyéctalo en el `Cliente` (Agregación). Por último, el `Cliente` entra a la `PlataformaCentro` y ocupa uno de los espacios (Composición en acción). Si todo funciona y no hay nulos, has conquistado el diseño orientado a objetos.